

# Day 5 Report — AI Internship

## Topic: Support Vector Machines (SVM), Kernel Methods & Text Classification using Naive Bayes

**Name:** Shafiq ur Rehman

**Day:** 5

**Date:** 07 June 2026

**Technology Stack:** Python, Scikit-learn, Pandas, Matplotlib, TF-IDF

---

### Objective

The objective of Day 5 was to:

- Understand Support Vector Machines (SVM) and kernel methods
  - Train and compare different SVM kernels (Linear, RBF, Polynomial)
  - Visualize decision boundaries for classification
  - Build a text classification system using TF-IDF and Naive Bayes
  - Evaluate model performance using precision, recall, and F1-score
- 

### Part 1: Support Vector Machine (SVM)

#### ◆ Concept Understanding

Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification. It works by finding the optimal hyperplane that separates different classes with the maximum margin.

Key concepts learned:

- **Hyperplane:** Decision boundary separating classes
- **Support Vectors:** Data points closest to the boundary
- **Margin:** Distance between classes

- **Kernel Trick:** Technique to transform data into higher dimensions for non-linear separation
- 

## ◆ Kernels Studied

### 1. Linear Kernel

Used when data is linearly separable.

### 2. RBF Kernel (Radial Basis Function)

Used for non-linear data; creates flexible decision boundaries.

### 3. Polynomial Kernel

Uses polynomial transformation for classification.

---

## ◆ Implementation Summary

- Dataset: Iris dataset (150 samples, 3 classes)
  - Features: 4 numerical features
  - Train-test split applied
  - Models trained using Scikit-learn SVC
- 

## ◆ Results

| Model          | Accuracy |
|----------------|----------|
| Linear SVM     | 1.00     |
| RBF SVM        | 1.00     |
| Polynomial SVM | 1.00     |

---

## ◆ Observation

All kernels achieved 100% accuracy due to the simplicity and linear separability of the Iris dataset. The Linear kernel was sufficient for classification, while RBF and Polynomial kernels did not significantly improve performance.

---

## ◆ Decision Boundary Visualization

A 2D decision boundary was plotted using two features (sepal length and sepal width). The plot showed:

- Clear separation of Setosa class
  - Partial overlap between Versicolor and Virginica
  - Linear hyperplanes dividing feature space
- 

# Part 2: Text Classification using Naive Bayes

## ◆ Concept Understanding

Naive Bayes is a probabilistic classification algorithm based on Bayes' Theorem. It assumes independence between features and calculates the probability of each class.

Used for:

- Spam detection
  - Sentiment analysis
  - Document classification
- 

## ◆ TF-IDF Vectorization

Text data was converted into numerical features using TF-IDF:

- **TF (Term Frequency):** Frequency of words in a document
- **IDF (Inverse Document Frequency):** Importance of rare words

This helps highlight important words while reducing weight of common words.

---

## ◆ Model Used

- Algorithm: Multinomial Naive Bayes
- Dataset: SMS Spam Collection
- Classes: Spam / Ham

---

## ◆ Evaluation Metrics

| Metric    | Score  |
|-----------|--------|
| Accuracy  | 0.9668 |
| Precision | 1.00   |
| Recall    | 0.7533 |
| F1 Score  | 0.8593 |

---

## ◆ Observation

- The model achieved perfect precision, meaning no normal messages were incorrectly classified as spam.
- Recall was lower, indicating some spam messages were missed.
- Overall performance is strong but can be improved by tuning thresholds or using advanced models.

---

## Key Learnings

- SVM constructs optimal decision boundaries using margin maximization
  - Kernel trick enables SVM to handle non-linear data
  - TF-IDF is effective for converting text into numerical form
  - Naive Bayes performs well for text classification tasks
  - Precision and recall provide deeper insight than accuracy alone
-



## Tools & Libraries Used

- Python
  - Scikit-learn (SVC, MultinomialNB, TF-IDF Vectorizer)
  - Pandas
  - Matplotlib
  - NumPy
- 



## Conclusion

Day 5 successfully covered both geometric classification (SVM) and probabilistic text classification (Naive Bayes). The practical implementation reinforced understanding of kernel methods, NLP preprocessing, and model evaluation metrics. The SMS spam classifier demonstrated a complete machine learning pipeline from raw text to prediction.